Assistant RAG – Altéa Services

Documentation du projet : ce qui a été construit, étape par étape

Auteur : LAMIA Oummad

Dépôt GitHub :
https://github.com/LAMIAomd/TP-RAG-en-entreprise---Etapes-de-mise-en-place-d-un-RAG-Professionnel..git

En bref

Altéa Services est une entreprise fictive. Ses salariés posent souvent les mêmes questions (télétravail, congés, VPN, notes de frais…), mais les réponses sont dispersées dans **15 documents** internes (PDF et fichiers texte).

Le projet construit un **assistant** qui répond à ces questions à partir des documents, **en indiquant toujours de quel document vient la réponse**. Il fonctionne entièrement sur l'ordinateur, sans envoyer de données sur Internet.

> **En une image :** imaginez un bibliothécaire très rapide. Il retrouve d'abord les bonnes pages dans le classeur (étapes 1 à 5), puis quelqu'un rédige la réponse en lisant **uniquement** ces pages (étape 6). Ensuite, on vérifie que l'ensemble marche bien (étape 7) et on le met à disposition des salariés (étape 8). Cette méthode s'appelle un **RAG**.

Les étapes du projet

Étape	Ce qu'on fait, en une phrase	Fichier
0. Préparation	Installer les outils et créer les dossiers	–
1. Collecte	Lire les documents et en extraire le texte	`etape1_collecte.py`
2. Nettoyage	Enlever les espaces en trop et les mots coupés	`etape2_nettoyage.py`
3. Découpage	Couper le texte en petits morceaux (chunks)	`etape3_decoupage.py`
4. Indexation	Transformer chaque morceau en nombres et le ranger dans une base	`etape4_indexation.py`
5. Recherche	Retrouver les morceaux les plus proches de la question	`etape5_recherche.py`
6. Génération	Faire rédiger la réponse par l'IA à partir de ces morceaux	`etape6_generation.py`
7. Évaluation	Mesurer avec des chiffres si l'assistant se trompe	`etape7_evaluation.py`
8. Mise à disposition	Proposer une page web et publier le projet sur GitHub	`app.py`, `README.md`

Les étapes 1 à 4 sont faites **une seule fois** (préparer les documents). Les étapes 5 et 6 sont refaites **à chaque question**.

Mini-lexique

Mot	Ce que ça veut dire, simplement
RAG	Méthode : on cherche d'abord les passages utiles dans les documents, puis on demande à une IA de répondre à partir d'eux.
LLM	L'IA qui rédige les réponses. Ici, `llama3.2`, qui tourne sur l'ordinateur grâce au logiciel Ollama.
Chunk	Un petit morceau de texte (ici environ 500 caractères).
Embedding / vecteur	Une liste de 384 nombres qui représente le **sens** d'un texte. Deux textes de sens proche ont des vecteurs proches.
Base vectorielle	Le « classeur » qui range les vecteurs et retrouve vite les plus proches d'une question (ici ChromaDB).
k	Le nombre de morceaux récupérés pour répondre à une question (3 par défaut).
Score	Une note de ressemblance entre un morceau et la question : plus elle est proche de 1, plus le morceau est pertinent.
Prompt	Le texte envoyé à l'IA : une consigne, les extraits trouvés, puis la question.
Hallucination	Quand l'IA invente une réponse fausse mais dite avec assurance.

Étape 0 : Préparer l'environnement

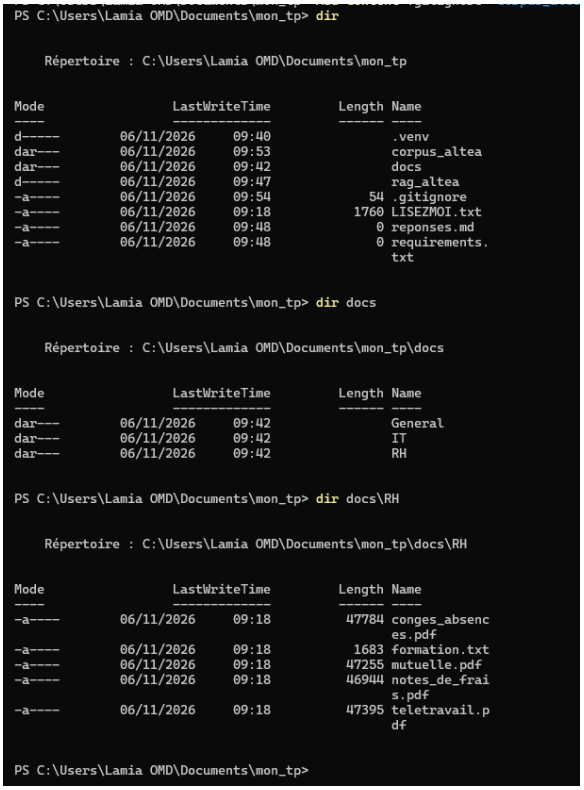
> **En simple :** avant d'écrire du code, on installe les outils et on vérifie qu'ils marchent.

Ce qui a été fait

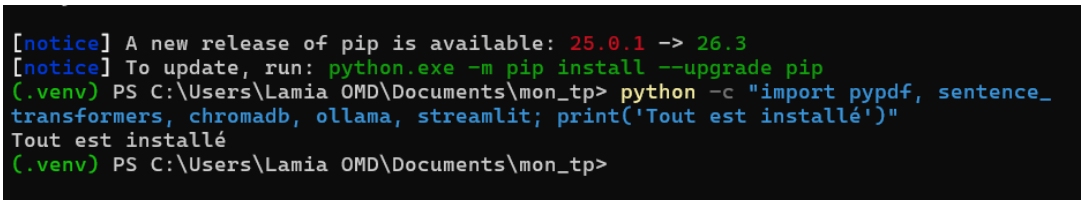
- **Vérification des outils** dans le terminal : Python 3.12.10, Git, et Ollama (le logiciel qui fait tourner l'IA sur l'ordinateur).
- **Test de l'IA** : on lance `ollama run llama3.2` et on lui demande la météo à Paris. Elle répond qu'elle n'a pas accès aux informations en temps réel. Le test prouve que l'IA fonctionne, et qu'**une IA seule ne connaît pas les documents d'Altéa** : c'est pour cela qu'on lui fournira des extraits.
- **Création du dossier de travail** `mon_tp` et d'un environnement virtuel `.venv` (une « boîte » Python réservée au projet).
- **Installation des bibliothèques** dans cette boîte. Le message « Tout est installé » confirme que tout est prêt.

Outil	À quoi il sert dans le projet
`pypdf`	Lire les fichiers PDF
`sentence-transformers`	Transformer les textes en vecteurs (embeddings)
`chromadb`	Stocker les vecteurs et chercher les plus proches
`ollama`	Faire tourner l'IA (llama3.2) sur l'ordinateur
`streamlit`	Créer la page web de l'assistant





À gauche : vérification des outils et test de llama3.2. À droite : dossiers du projet (docs/ contient General, IT et RH).



Contrôle final : toutes les bibliothèques s'importent sans erreur.

À retenir : les documents sont rangés par service dans `docs/RH`, `docs/IT` et `docs/General`, et chaque script du projet ira dans `rag_altea/`.

Étape 1 : Collecte

> **En simple :** le programme ouvre chaque document et en copie le texte, page par page.

Ce qui a été fait

- Le script `etape1_collecte.py` parcourt tout le dossier `docs/`. Pour un PDF, il lit chaque page avec `pypdf` ; pour un fichier texte, il lit le fichier entier (une seule page).
- Pour chaque page, il garde trois informations en plus du texte : le **nom du fichier**, le **numéro de page** et le **service** (RH, IT ou General).
- Résultat : **23 pages** lues, pour **15 documents** (8 PDF de 2 pages et 7 fichiers texte d'une page).

```
(.venv) PS C:\Users\Lamia OMD\Documents\mon_tp> & 'c:\Users\Lamia OMD\Documents\mon_tp\.venv\Scripts\python.exe' '"c:/Users/Lamia OMD/Documents/mon_tp/rag_altea/etape1_collecte.py"'
23 pages lues
horaires.txt : 1 page(s)
livret_accueil.pdf : 2 page(s)
plan_acces.pdf : 2 page(s)
reglement_interieur.pdf : 2 page(s)
restauration.txt : 1 page(s)
charte_informatique.pdf : 2 page(s)
demande_materiel.txt : 1 page(s)
mot_de_passe.txt : 1 page(s)
securite_emails.txt : 1 page(s)
vpn.txt : 1 page(s)
conges_absences.pdf : 2 page(s)
formation.txt : 1 page(s)
mutuelle.pdf : 2 page(s)
```

Le script affiche le nombre total de pages puis le nombre de pages de chaque document.

À retenir : on garde le nom du fichier, la page et le service pour pouvoir **citer la source** dans la réponse et **filtrer** la recherche par service.

Étape 2 : Nettoyage

> **En simple :** le texte copié depuis un PDF est un peu « sale » ; on le remet au propre avant de le découper.

Ce qui a été fait

Trois règles de nettoyage sont appliquées à chaque page :

- plusieurs espaces d'affilée deviennent **un seul espace** ;
- trois sauts de ligne ou plus deviennent **une seule ligne vide** ;
- les mots coupés en fin de ligne sont **recollés** (« télé-travail » devient « télétravail »).

Les pages devenues vides sont supprimées. Ici, on passe de **23 pages à 23 pages** : aucune page n'était vide.

```
(.venv) PS C:\Users\Lamia OMD\Documents\mon_tp> & 'c:\Users\Lamia OMD\Documents\mon_tp\.venv\Scripts\python.exe' '"c:/Users/Lamia OMD/Documents/mon_tp/rag_altea/etape2_nettoyage.py"'
Pages avant nettoyage : 23
Pages après nettoyage : 23

===== AVANT : livret_accueil.pdf page 1 =====
"Altéa Services — Document interne — Diffusion restreinte\n1\nDocument fictif réalisé à des fins pédagogiques\nLivret d'accueil du nouveau salarié\nGEN-002 — édition 2026\nBienvenue chez Altéa Services\nAltéa Services est une entreprise fictive de services aux entreprises, fondée en 2004, qui\ncompte environ 1 400 salariés sur trois sites : Paris (siège), Lyon et Nantes. Ce livret\nrassemble ce qu'il faut savoir pour bien démarrer.\nVotre premier jour\nVous êtes accueilli à 9 h 30 par votre manager et un m"

===== APRES =====
"Altéa Services — Document interne — Diffusion restreinte\n1\nDocument fictif réalisé à des fins pédagogiques\nLivret d'accueil du nouveau salarié\nGEN-002 — édition 2026\nBienvenue chez Altéa Services\nAltéa Services est une entreprise fictive de services aux entreprises, fondée en 2004, qui\ncompte environ 1 400 salariés sur trois sites : Paris (siège), Lyon et Nantes. Ce livret\nrassemble ce qu'il faut savoir pour bien démarrer.\nVotre premier jour\nVous êtes accueilli à 9 h 30 par votre manager et un m"
(.venv) PS C:\Users\Lamia OMD\Documents\mon_tp>
```

Une même page affichée avant et après nettoyage (page 1 de livret_accueil.pdf).

À retenir : il reste un défaut : chaque page PDF commence par un en-tête répété (« Altéa Services – Document interne… ») qui n'apporte rien. On pourrait le supprimer avec une règle supplémentaire.

Étape 3 : Découpage

> **En simple :** on coupe le texte en petits morceaux, comme des fiches. Un morceau reprend la fin du précédent pour ne pas couper une idée en deux.

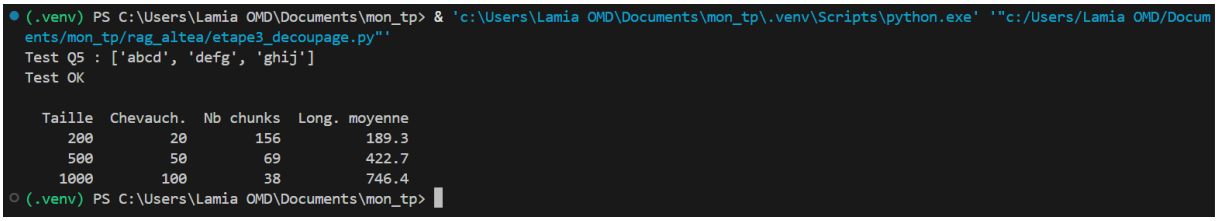
Pourquoi découper ?

Un document entier est trop long pour être envoyé à l'IA, et la recherche est plus précise sur de petits passages. Chaque morceau (**chunk**) a un identifiant unique, par exemple `vpn.txt-p1-0`.

Ce qui a été fait

- **Test de la fonction** : `decouper("abcdefghij", taille=4, chevauchement=1)` renvoie `["abcd", "defg", "ghij"]`. Chaque morceau reprend la dernière lettre du précédent : c'est le chevauchement.
- **Trois réglages comparés** sur tout le corpus (voir le tableau).
- **Réglage retenu : 500 caractères avec 50 de chevauchement**, soit 69 chunks.

Taille (caractères)	Chevauchement	Nombre de chunks	Longueur moyenne
200	20	156	189,3
500 (retenu)	**50**	**69**	**422,7**
1000	100	38	746,4



Le test « Test OK » valide la fonction ; le tableau compare les trois tailles.

À retenir : trop petit, un morceau perd son contexte ; trop grand, il mélange plusieurs sujets. 500 caractères est un bon compromis pour ces documents.

Étape 4 : Embeddings et indexation

> **En simple :** on transforme chaque morceau de texte en une liste de nombres qui représente son sens, puis on range ces listes dans une base pour retrouver vite les plus proches d'une question.

Partie A : découvrir les embeddings

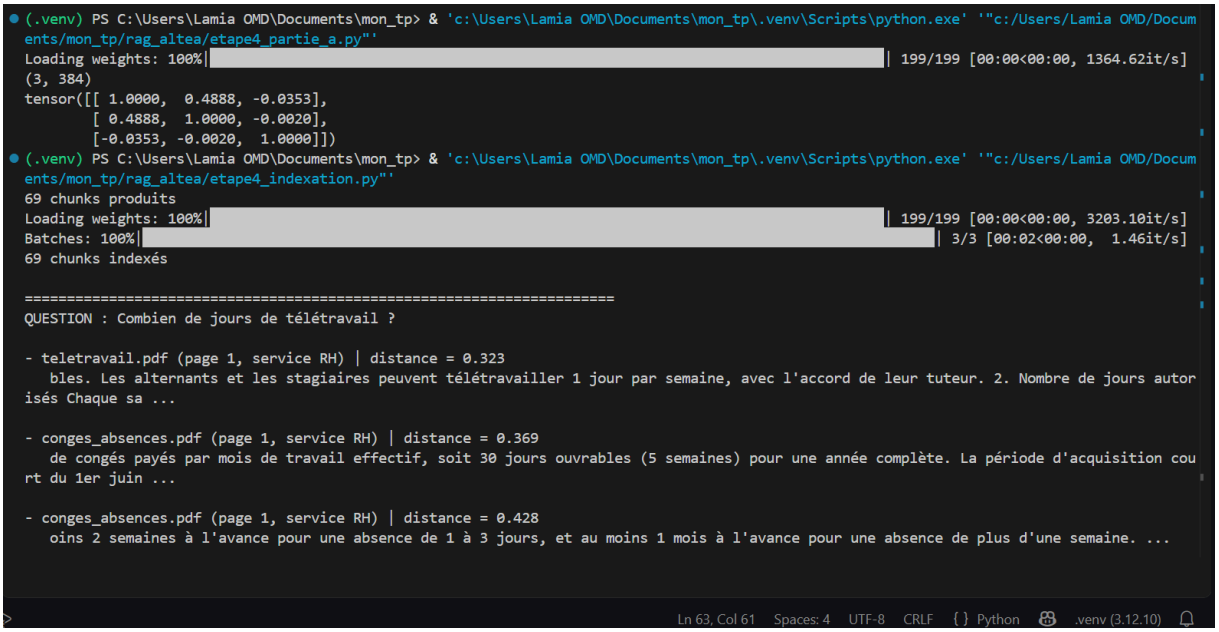
Le modèle `paraphrase-multilingual-MiniLM-L12-v2` (il comprend le français) transforme chaque phrase en un vecteur de **384 nombres**. Test avec trois phrases :

- « Comment poser un jour de congé ? » et « Quelle est la procédure pour demander des vacances ? » : similarité de **0,49**. Elles n'ont presque aucun mot en commun, mais le modèle comprend qu'elles ont le **même sens**.
- « Mon ordinateur ne se connecte pas au VPN. » : similarité proche de **0** avec les deux autres, car le sujet est sans rapport.

Partie B : ranger les chunks dans la base

- Les 69 chunks sont transformés en vecteurs puis rangés dans **ChromaDB**, dans le dossier `base_vectorielle`, avec leur fichier, leur page et leur service.

- On utilise `upsert` (« ajouter ou mettre à jour ») : relancer le script ne crée pas de doublons.
- Contrôle : **69 chunks produits, 69 chunks indexés**.
- Premier essai de recherche : pour « Combien de jours de télétravail ? », le morceau le plus proche vient de `teletravail.pdf` (distance 0,323), devant `conges_absences.pdf`.



En haut : test des embeddings. En dessous : indexation des 69 chunks et première recherche.

À retenir : une **distance** faible veut dire « proche de la question » (le score de l'étape 5 vaut 1 moins la distance).

Étape 5 : Recherche

> **En simple :** quand un salarié pose une question, on la transforme en nombres avec le même modèle, puis on récupère les morceaux dont les nombres ressemblent le plus.

Comment ça marche

- Le salarié pose une question, par exemple « Combien de jours de télétravail ? ».
- La question devient un vecteur, avec le **même modèle** que pour les documents.
- La base compare ce vecteur à ceux de tous les chunks et renvoie les **k plus proches** (3 par défaut).
- Pour chaque résultat, on garde le texte, le fichier, la page, le service et un **score**.

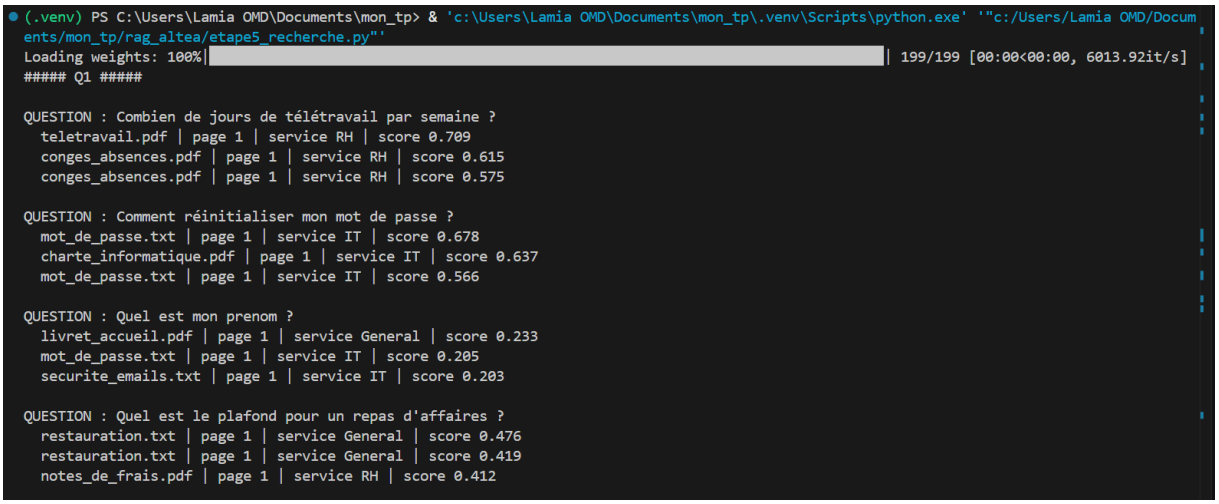
Deux réglages

- **k** : combien de morceaux on récupère. Plus k est grand, plus on a de chances de trouver la réponse, mais plus il y a de bruit.
- **service** : on peut limiter la recherche à RH, IT ou General.

Résultats obtenus

Question	Meilleur résultat	Score
Combien de jours de télétravail par semaine ?	teletravail.pdf, page 1	0,709
Comment réinitialiser mon mot de passe ?	mot_de_passe.txt, page 1	0,678

Question	Meilleur résultat	Score
Quel est le plafond pour un repas d'affaires ?	restauration.txt, page 1	0,476
Quel est mon prénom ? (hors sujet)	livret_accueil.pdf, page 1	0,233



Source, page, service et score des 3 meilleurs morceaux pour chaque question.

À retenir : pour une question bien couverte, le score dépasse 0,67 ; pour une question sans rapport (« Quel est mon prénom ? »), il reste autour de 0,2. Le score donne donc une idée de la pertinence.

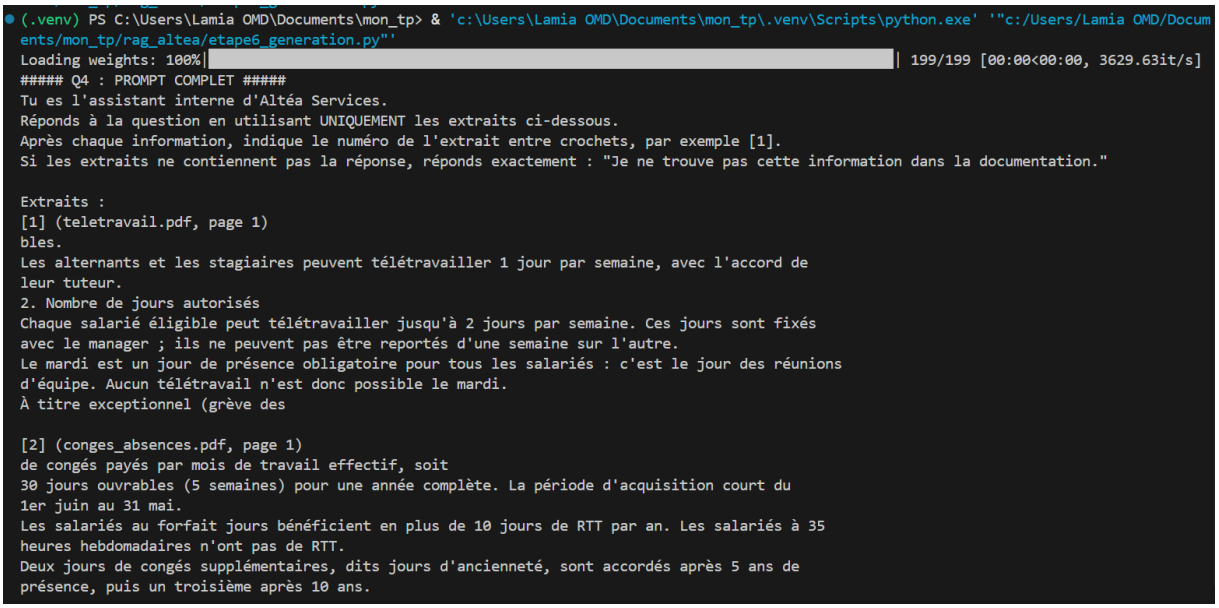
Étape 6 : Génération

> **En simple :** on donne à l'IA les morceaux trouvés et on lui demande de répondre **uniquement avec eux**, en citant ses sources.

Le prompt : trois parties

- **La consigne** : « Tu es l'assistant interne d'Altéa Services. Réponds uniquement avec les extraits. Cite le numéro de l'extrait, par exemple [1]. »
- **Les extraits** : les 3 morceaux trouvés, numérotés, avec leur fichier et leur page.
- **La question** du salarié.

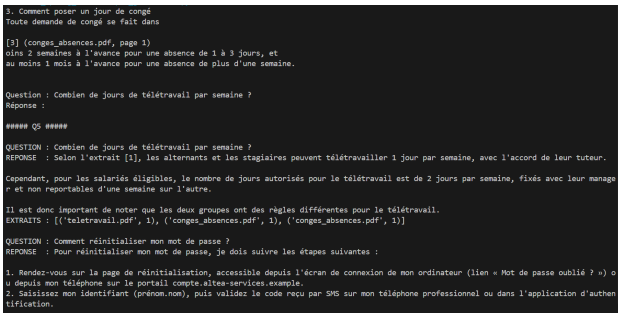
Deux protections sont ajoutées : une **phrase de repli** (« Je ne trouve pas cette information dans la documentation. ») quand les extraits ne contiennent pas la réponse, et `temperature=0` pour obtenir des réponses stables et fidèles, sans « créativité ».

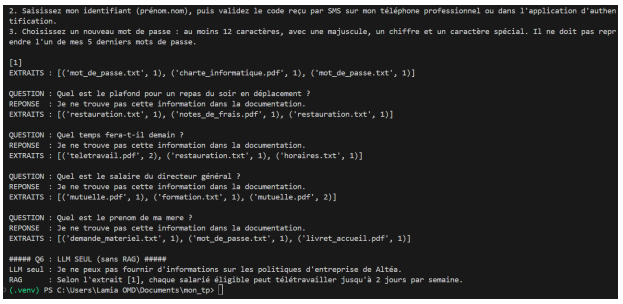


Le prompt complet pour la question « Combien de jours de télétravail par semaine ? ».

Résultats obtenus

- Les questions avec réponse dans les documents (télétravail, mot de passe) reçoivent une réponse **avec les sources**.
- Les questions sans réponse (« Quel temps fera-t-il demain ? », « Quel est le salaire du directeur général ? ») déclenchent bien la **phrase de repli** : l'assistant n'invente rien.
- **Avec ou sans RAG** : posée seule, l'IA répond qu'elle ne peut pas parler des politiques d'Altéa. Avec le RAG, elle répond que le salarié éligible peut télétravailler jusqu'à 2 jours par semaine.





Réponses de l'assistant aux questions de test, puis comparaison de l'IA seule avec le RAG.

À retenir : le RAG limite les **hallucinations** parce que l'IA s'appuie sur des extraits réels, et la phrase de repli remplace l'invention quand l'information manque.

Étape 7 : Évaluation

> **En simple :** un assistant d'entreprise doit être mesuré avant d'être utilisé. On pose 10 questions dont on connaît la réponse et on compte les erreurs.

Mesure 1 : la recherche trouve-t-elle le bon document ?

Pour chacune des 10 questions du fichier `evaluation.csv`, on regarde si le document attendu est parmi les k résultats.

Valeur de k	Questions où le bon document est trouvé
k = 1	90 %
k = 3	**100 %**
k = 5	100 %

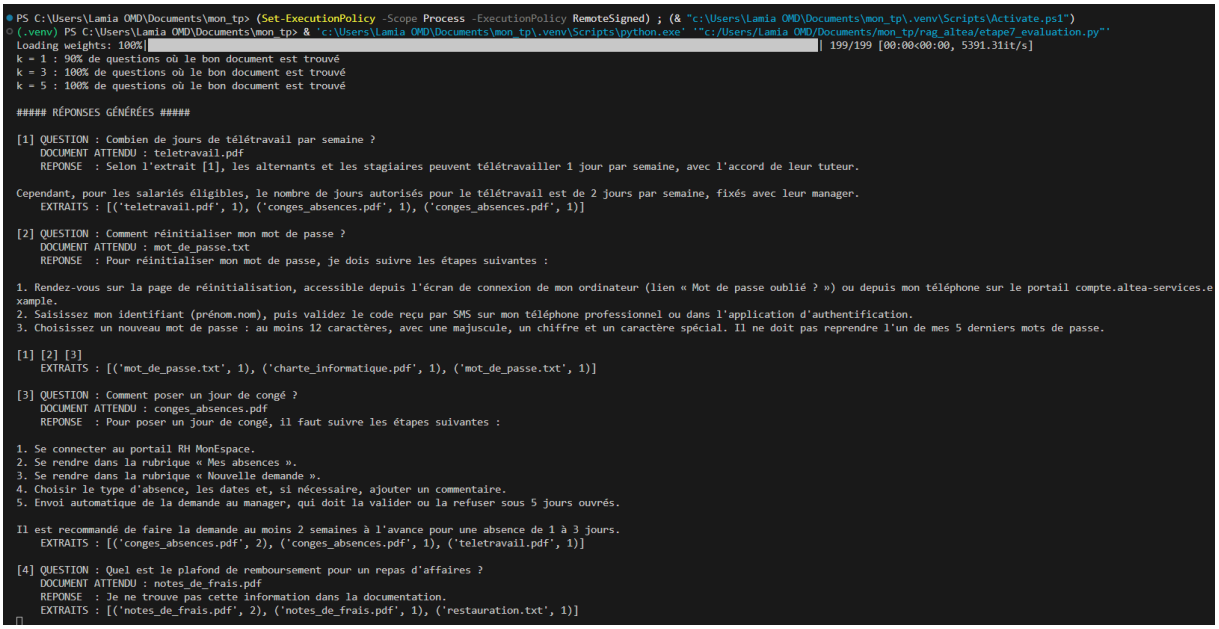
Avec k = 1, une seule question est ratée : « Comment adhérer à la mutuelle ? » (un autre document passe devant). À partir de k = 3, le bon document est toujours trouvé.

Mesure 2 : les réponses sont-elles justes ?

Les 10 réponses générées ont été relues à la main :

- **8 réponses justes sur 10 (80 %)**, 1 partielle, 1 fausse ;
- **6 réponses sur 10 (60 %)** citent correctement leur source avec un numéro [n].

La réponse fausse concerne le plafond d'un repas d'affaires : l'assistant répond « Je ne trouve pas cette information » alors que le bon document (`notes_de_frais.pdf`) avait été trouvé. L'erreur vient donc de la **génération**, pas de la recherche.



Taux de réussite de la recherche, puis réponses générées pour les questions de test.

À retenir : la recherche fonctionne très bien (100 % dès k = 3) ; les erreurs restantes viennent surtout de l'IA locale, un petit modèle parfois trop prudent.

Étape 8 : Mise à disposition et GitHub

> **En simple :** on crée une page web où un salarié tape sa question et lit la réponse, puis on publie le projet sur GitHub pour qu'il soit documenté et réutilisable.

L'interface web

Le fichier `app.py` utilise **Streamlit**. Il appelle la fonction `repondre` de l'étape 6, qui enchaîne la recherche (étape 5), la construction du prompt et la génération. La page affiche :

- un champ « Votre question » ;

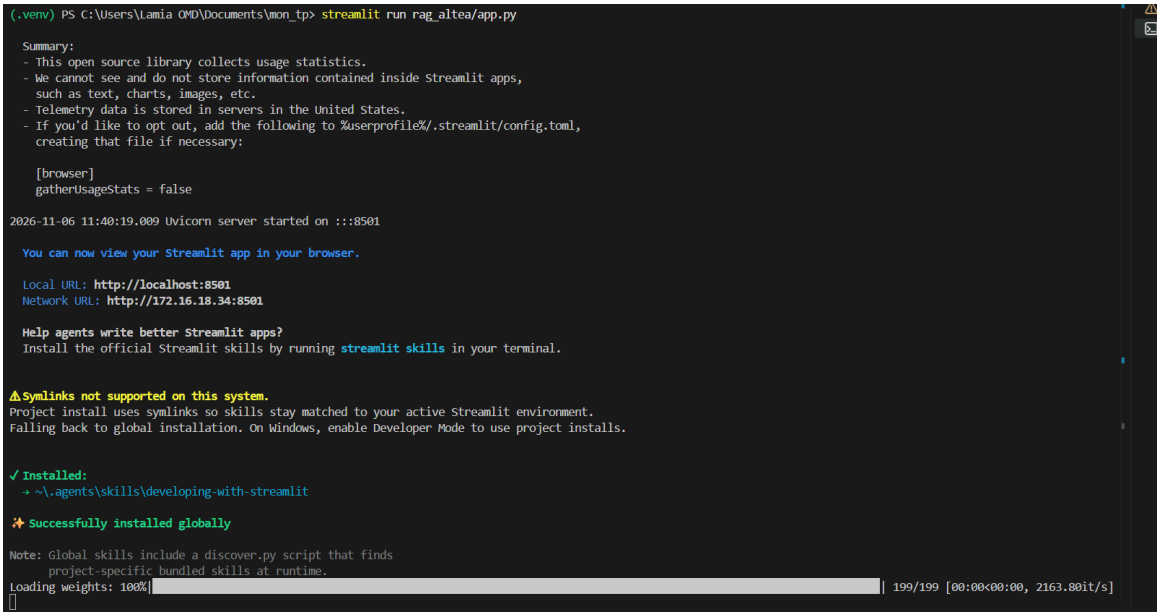
- un indicateur de chargement pendant la recherche ;
- la réponse ;
- une rubrique **Sources** : un bloc dépliable « [i] fichier, page X » par extrait, qui montre le texte du passage. Le salarié peut ainsi vérifier d'où vient l'information.

Lancer l'assistant

Depuis le dossier `mon_tp`, avec Ollama actif :

```
streamlit run rag_altea/app.py
```

La page s'ouvre dans le navigateur à l'adresse http://localhost:8501.



Le terminal confirme que l'application tourne (« Local URL : http://localhost:8501 »).



L'assistant répond à « combien de jours de télétravail pour un alternant ? » et affiche ses trois sources.

Publication sur GitHub

Le projet est publié ici :
https://github.com/LAMIAomd/TP-RAG-en-entreprise---Etapes-de-mise-en-place-d-un-RAG-Professionnel..git

- Le dépôt contient le code (`rag_altea/`), les documents (`docs/`), le `README.md`, le `requirements.txt` et les réponses du TP (`reponses.md`).
- Les dossiers `.venv/` et `base_vectorielle/` ne sont **pas** envoyés (fichier `.gitignore`) : ils se recréent facilement.
- La version livrée est marquée par le tag `v1.0`.

Refaire fonctionner le projet sur un autre ordinateur

- Cloner le dépôt, puis installer les bibliothèques : `pip install -r requirements.txt`.
- Télécharger l'IA : `ollama pull llama3.2`.
- Recréer la base : `python rag_altea/etape4_indexation.py`.
- Lancer l'assistant : `streamlit run rag_altea/app.py`.